

PROJECT A.R.T.E.M.I.S

Adaptive Reasoning, Task Execution and Multi-agent Intelligence System

Final Year Project Proposal | Department of Computer Science | FAST NUCES

Hira Khalid (23L-2594) | Doureesha Batool (23L-2651) | Hashir Rauf (23L-2572)

1. Problem

People who manage work spread across many files lose real time on tasks that have nothing to do with the work itself. This includes students, freelancers, and developers. A person steps away from a project for a day or more and returns without a clear memory of which files they were using, what they had already decided, or what they intended to do next. Folders accumulate unsorted files. The same manual chore gets repeated many times without ever being automated, because setting up automation usually takes more effort than doing the task by hand. Existing AI assistants, including Siri, Copilot, and current computer use agents from Microsoft, Google, OpenAI, and Anthropic, either require an explicit command every time or operate with broad, unbounded access to a user's entire system. Neither approach solves the specific problem of resuming interrupted work with a small, trustworthy memory of what was already done, while still supporting the everyday actions that kind of work actually requires, such as looking something up, sending a message, or drafting a document.

2. Solution

ARTEMIS is a bounded, opt in desktop assistant. A user selects a folder and adds it as a workspace. ARTEMIS only ever knows about files, tabs, and notes explicitly added to a workspace, and does not read anything outside a chosen folder. Nothing is ever deleted, and no action that changes a file or leaves the application, such as sending an email, happens without the user seeing it first and approving it, with an undo available afterward. A single panel, called What ARTEMIS Knows, lists everything the system currently holds, with a delete option next to each item. This design is intentionally small. It trades the ambition of a full operating system copilot for something that is fast to build, easy to trust on first use, and possible to demonstrate completely within a two semester timeline.

3. Features

The first version consists of nine features, built in this order.

- 1. Pick Up Where You Left Off:** Shows the files recently changed in a workspace, the last saved notes, and what the user said they were about to do next. One click reopens any file. No AI interpretation is used for this feature.
- 2. Tidy Up a Folder:** Reads file names and a short text extract from each document, then proposes a grouping as a preview. Nothing moves until the user approves. A single undo restores everything.
- 3. Pull Together Notes From a Few Documents:** Combines citations or key points from a small, explicitly selected set of documents into a new file. Existing files are never overwritten.
- 4. Notice a Repeated Task:** If the same manual pattern happens three times or more, ARTEMIS asks whether to automate it. It never acts without asking first, and every automatic run can be undone.
- 5. Code Assistant:** Reads the source files already inside a workspace and answers direct questions, including explaining errors. A suggested fix is shown as a diff and applied only after approval. It does not write or run code on its own.
- 6. Web Search:** Looks up information on the open web when a user explicitly asks a question that a workspace cannot answer on its own, such as finding a missing reference or checking a current fact. This is a direct, on demand action, not background browsing, and no site visited is tracked beyond the single request.
- 7. Mailing:** Drafts an email using content already available in the workspace, such as a summary or a finished document. The full draft is shown for review, and the email is sent only after the user approves it. Every sent message is recorded in the activity log.
- 8. Document Drafting:** Writes a first draft of a document, such as a report, cover letter, or summary, based on material already in the workspace and a short description of what is needed. The draft is saved as a new file for the user to edit; nothing is finalized or sent automatically.

9. Speech Input and Output: Lets a user speak a request instead of typing it, and hear responses read aloud instead of reading them. This is an alternate way to interact with every other feature, not a separate capability, and follows the same approval steps before any action is taken.

A tenth feature, a workspace change summary showing what changed since a user last checked, is planned as a stretch goal for a later iteration rather than a first version requirement.

4. Pain Points Addressed

Pain Point	How ARTEMIS Addresses It
Lost time reconstructing work after a break	Feature 1 restores context in seconds instead of manual searching
Messy, unsorted folders	Feature 2 groups files with a reversible, approval based action
Scattered notes and sources across documents	Feature 3 consolidates them into one new, citation grounded file
Repetitive manual file handling	Feature 4 detects the pattern and offers automation, never imposes it
Getting stuck on unfamiliar or broken code	Feature 5 explains the issue and proposes a reviewable fix
Needing outside information a workspace does not contain	Feature 6 answers a direct question on demand, without ongoing browsing
Time spent writing routine emails	Feature 7 drafts the message and sends it only after approval
Starting a document from a blank page	Feature 8 produces an editable first draft from existing material
Typing and reading being inconvenient in some situations	Feature 9 offers a voice based alternative to the same approval steps
Distrust of AI tools with broad system access	The workspace model and preview approve undo pattern bound every action

5. Research Objectives

1. **O1.** Design a bounded, opt in workspace model that tracks only explicitly shared files, tabs, and notes, with a fully visible audit of what the system knows.
2. **O2.** Build and evaluate a session resume feature that reduces the time needed to reconstruct interrupted work.
3. **O3.** Build a preview approve undo interaction pattern for file organization and repeated task automation, and measure its effect on user trust and control.
4. **O4.** Build a small scale, citation grounded document synthesis feature and measure its accuracy where source metadata is available.
5. **O5.** Build a bounded code assistant that explains and proposes fixes for code within a workspace without autonomous execution.
6. **O6.** Build an on demand information lookup feature that answers a direct question without continuous or unrequested browsing.
7. **O7.** Build an approval gated mailing feature and measure how the extra review step affects user trust and time spent on routine correspondence.
8. **O8.** Build a document drafting feature and measure the quality and usefulness of a first draft produced from existing workspace material.
9. **O9.** Build a speech based input and output mode and evaluate whether it changes ease of use or task completion time relative to typed interaction.
10. **O10.** Evaluate the system for its effect on task completion time, cognitive workload, and user trust, using a study design appropriate to each feature being tested.

6. Hypothesis

A desktop assistant that is deliberately bounded to an explicitly shared workspace, rather than given broad access to a user's system, will be trusted and adopted more readily, while still measurably reducing the time and effort required to resume interrupted work, organize files, understand unfamiliar code, look things up, write routine messages, and produce first drafts of documents. Restricting scope and requiring approval for every action,

including the ones that reach outside the workspace such as web search and mailing, is expected to improve, not limit, real world usefulness, because it removes the main reason users abandon AI assistants that touch their files or act on their behalf: the fear of losing control over their own data and communication.

7. Role of Agentic AI

ARTEMIS is built as an agentic AI system, not a single prompt-response assistant. Each user request is handled by an agent that reasons about what the request needs, chooses which of the available tools to use, such as reading a file, searching the web, or drafting an email, and decides whether more than one step is required to complete the task. This differs from a standard chatbot because the agent plans a sequence of actions rather than producing one reply, and it differs from a fully autonomous agent because every action that changes a file or leaves the workspace stops for user approval before it executes. Agentic reasoning is what allows a single request, such as asking ARTEMIS to prepare a summary and email it to a supervisor, to be broken down automatically into the right sequence of tool calls, in Features 3, 6, 7, and 8 specifically, while the approval and undo model described in Section 2 keeps that autonomy bounded and reviewable at every step.

8. References

11. Anthropic. Claude computer use and Cowork agent documentation, 2026.
12. Gonzalez, V. M., and Mark, G. Constant, constant, multitasking craziness: managing multiple working spheres. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2004.
13. Google. Gemini computer use API documentation, 2026.
14. Hevner, A. R., March, S. T., Park, J., and Ram, S. Design science in information systems research. MIS Quarterly, 28(1), 2004.
15. Iqbal, S. T., and Horvitz, E. Disruption and recovery of computing tasks: field study, analysis, and directions. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2007.
16. Microsoft. Copilot Studio computer use agents, general availability, May 2026.
17. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: synergizing reasoning and acting in language models. International Conference on Learning Representations, 2023.